INTERVIEW STUDY GUIDE

Blog Writing Agent

Architecture, design decisions, trade-offs and interview Q&A
LangGraph · Pydantic · Tavily · Gemini image generation · Streamlit

Based on `bwa_backend.py` (574 lines) and `bwa_frontend.py` (468 lines).
Line references like **L150–171** point to those files. Audience: intern / fresher interviews.

> **THE ONE IDEA TO REMEMBER**
> AI can write the lines of code. **You** decide the architecture: what is an LLM call, what is plain Python, what is enforced in code instead of requested in a prompt, and how failures are handled. Interviewers are testing whether you can explain *why* for every box in the diagram.

0. How to use this guide

1	The big picture: pipeline, state, data flow	**7**	Grounding, prompts and safety
2	Concepts primer and glossary	**8**	Providers, cost and reliability
3	LangGraph in depth	**9**	Streamlit frontend
4	Node-by-node walkthrough	**10**	Weak points (honest limitations)
5	Schemas and structured output	**11**	Improvements and debugging stories
6	The image pipeline	**12–13**	Interview Q&A bank, cheat sheet

Suggested order. Read 1–2 first (the story), 3–6 for depth, then 10–11 so you can discuss weaknesses calmly. Finally drill section 12 out loud. Cover the answers and say yours first.

The 30-second pitch

> **SAY IT LIKE THIS**
> "I built an AI agent that writes a technical blog from a single topic. It first decides whether it needs fresh web research, plans the outline as structured data, then writes every section in parallel using LangGraph's fan-out. A reducer merges the sections, a second LLM step decides where diagrams would help, an image model generates them, and Python inserts them into the Markdown. A Streamlit app lets you run it, watch progress, preview and download the result."

The 2-minute version (add these points)

- **Three modes** (closed_book / hybrid / open_book) so the agent only pays for web search when freshness matters.
- **Plan first, then write:** a Pydantic `Plan` is the contract between the planner and the writers, which makes parallel writing possible.
- **Grounding:** in open_book mode a worker may only state events that it can back with a URL from the evidence list; otherwise it must say "Not found in provided sources."
- **Prompts ask, code enforces:** the recency window, the forced `blog_kind`, section ordering and file I/O are all done in Python, not left to the model.
- **Graceful degradation:** if an image fails to generate, the document still renders with a placeholder block containing the prompt.
- **What I would improve:** (name two weak points from section 10 and your fix).

Tech stack at a glance

Layer	Technology	Why it is used
Orchestration	LangGraph `StateGraph`, `Send`	Conditional routing, dynamic parallel fan-out, subgraphs, streaming of node updates
LLM access	LangChain `ChatGroq` (current), `ChatOpenAI` pointed at OpenRouter (earlier)	One interface for many providers; `with_structured_output`
Model	Groq `openai/gpt-oss-20b`, temperature 0.3	Fast, free-tier friendly; low temperature for consistent structured output
Schemas	Pydantic v2	Validation, JSON schema for function calling, safe `model_dump()` across graph boundaries
Web search	Tavily	Search API built for LLM use, returns cleaned page content
Image generation	Google Gemini `gemini-2.5-flash-image` via `google-genai`	Cheap, returns inline image bytes
Frontend	Streamlit + pandas	Python-only UI, quick to build, easy progress streaming
Config	`python-dotenv`	Keys in `.env`: `GROQ_API_KEY`, `TAVILY_API_KEY`, `GOOGLE_API_KEY_image`

1. The big picture

The system is a **directed graph of nine steps** that share one state object. A topic goes in; a Markdown file (plus an `images/` folder) comes out.

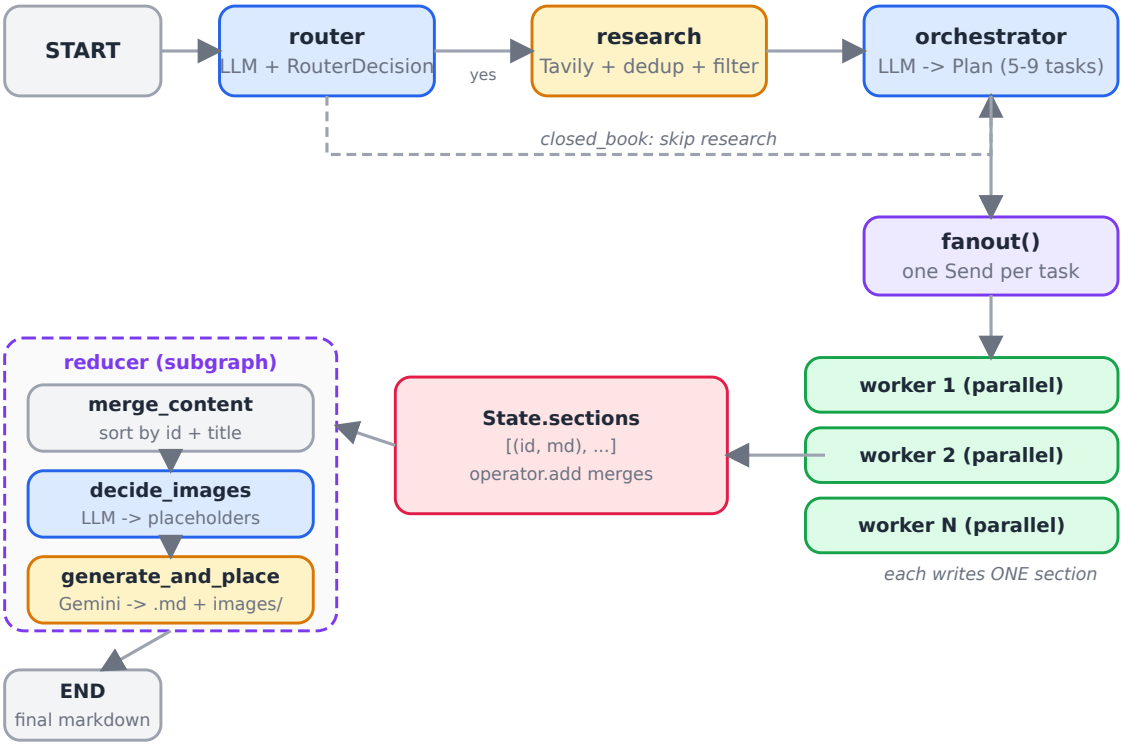


Figure 1. End-to-end pipeline. Solid arrows are normal edges, dashed arrows are conditional or optional paths.

What each step reads and writes

Memorise this table. It is the fastest way to show you understand the data flow. Each node returns only the keys it changes and LangGraph merges them into the shared state.

Node	Reads from state	Writes to state
`router`	topic, as_of	needs_research, mode, queries, recency_days
`research`	queries, mode, as_of, recency_days	evidence
`orchestrator`	topic, mode, evidence, as_of, recency_days	plan
`fanout` (edge function)	plan, evidence, topic, mode, as_of, recency_days	nothing; returns N `Send` packets
`worker` × N	its own payload (task, plan, evidence, ...)	sections (appended)
`merge_content`	plan, sections	merged_md
`decide_images`	merged_md, plan, topic	md_with_placeholders, image_specs
`generate_and_place_images`	md_with_placeholders, image_specs, plan	final (and files on disk)

The three execution paths

Mode	Chosen when	Research?	Recency window	Writing policy
closed_book	Evergreen concepts (e.g. “Self Attention”)	No, skip to orchestrator	3650 days (effectively none)	Free to explain from model knowledge
hybrid	Evergreen plus up-to-date tools, models, examples	Yes	45 days (set, but only enforced in open_book)	Sections marked `requires_citations` must cite evidence URLs
open_book	“latest”, weekly roundup, pricing, policy	Yes	7 days, **hard filter** in code	`blog_kind` forced to `news_roundup`; every claim needs an evidence URL

INTERVIEW TIP
The recency numbers (7 / 45 / 3650) are set in Python from the model's *mode* decision (L159–164), not generated by the LLM. Be ready to say why: policy constants should be deterministic and testable.

2. Concepts primer and glossary

Workflow vs agent (be precise, it impresses)

People use “agent” loosely. In strict terms, an **agent** lets the LLM decide in a loop which tool to call next (ReAct style). Your project is better described as an **agentic workflow**: a fixed graph that you designed, where LLM calls sit inside nodes and one LLM decision (the router) steers the control flow. You chose this on purpose: the task (write a blog) has known steps, so a workflow is cheaper, more predictable and easier to debug than a free-roaming agent.

> **SAY IT LIKE THIS**
> “It is an agentic workflow rather than an open-ended agent. I fixed the steps and let the LLM decide only where judgement is needed: routing, planning, writing and where images help.”

Ideas you must be able to explain in one or two sentences

Concept	Plain explanation and where it appears in your project
Structured output	Forcing the model to return data matching a schema (Pydantic) instead of free text. Used by router, orchestrator and image planner.
Function / tool calling	The schema is sent as a “tool” definition; the model replies with the arguments for that tool. This is how most providers implement structured output.
Grounding	Making the model rely on supplied sources rather than memory. Your evidence list plus the “only cite these URLs” rule.
Retrieve-then-generate (RAG-lite)	Fetch external context (Tavily), then generate with it. You use live web search, not a vector database.
Map-reduce	Split work into independent pieces (map: workers), then combine (reduce: reducer). Implemented with `Send` and an `operator.add` state key.
Conditional edge	A function that inspects state and chooses the next node. `route_next` chooses research vs orchestrator.
Reducer (LangGraph sense)	A function telling LangGraph how to merge several writes to the same state key. Not the same as your `reducer` node, which is a stage name. Say “state reducer” vs “reducer node” to avoid confusion.
Subgraph	A compiled graph used as a single node inside another graph. Your image-aware reducer.
Token, context window	Models read and write tokens (about 3/4 of a word). Input + output must fit the context window; output has its own cap (`max_tokens`).
Reasoning tokens	Hidden “thinking” tokens some models generate before answering. They count toward the completion limit. `gpt-oss-20b` is a reasoning model.
Rate limits (RPM / TPM)	Requests and tokens per minute allowed by the provider. Parallel workers can hit TPM quickly on free tiers.
Temperature	Randomness of sampling. 0.3 is low: more consistent structure, still some natural variation in prose.
Idempotency	Running the same step again does not duplicate work or harm. Image generation skips files that already exist (but see weak point 7).
Prompt injection	Untrusted text (web pages) containing instructions that try to hijack the model. Relevant because search snippets enter your prompts.

3. LangGraph in depth

3.1 What it is and why you picked it

LangGraph models an LLM application as a **graph**: nodes are Python functions, edges decide what runs next, and a shared **state** object carries data between nodes. Compared with a simple LangChain chain (a straight line) it adds the three things you needed:

- **Branching** (skip research for evergreen topics).
- **Dynamic parallelism** (one worker per section, where the number of sections is unknown until the plan exists).
- **Composition and observability** (a reducer subgraph; streaming of per-node updates to the UI).

Alternatives you can mention: a plain Python script with `asyncio.gather` (works, but you hand-roll state, routing and streaming), CrewAI or AutoGen (role-based multi-agent frameworks, more autonomy but less control), and an LLM "agent with tools" loop (flexible but harder to bound cost and behaviour).

3.2 State (L92–114)

```
class State(TypedDict):
    topic: str
    mode: str; needs_research: bool; queries: List[str]
    evidence: List[EvidenceItem]; plan: Optional[Plan]
    as_of: str; recency_days: int
    sections: Annotated[List[tuple[int, str]], operator.add]   # the special one
    merged_md: str; md_with_placeholders: str; image_specs: List[dict]
    final: str
```

- State is a `TypedDict`: a plain dict with declared keys. Each node receives the whole state and returns a **partial dict** of keys to update.
- By default an update **overwrites** the key. `sections` is different: `Annotated[..., operator.add]` attaches a **reducer function** that concatenates lists.
- **Why that matters:** several workers write `sections` in the same step. Without a reducer LangGraph cannot decide whose value wins and raises an error about multiple values per step for that key. With `operator.add`, `[a] + [b] + [c]` simply accumulates.

3.3 Edges and routing (L557–572)

```
g.add_edge(START, "router")
g.add_conditional_edges("router", route_next,
        {"research": "research", "orchestrator": "orchestrator"})
g.add_edge("research", "orchestrator")
g.add_conditional_edges("orchestrator", fanout, ["worker"])
g.add_edge("worker", "reducer"); g.add_edge("reducer", END)
app = g.compile()
```

- **Normal edge:** always go from A to B.
- **Conditional edge:** a function returns a label (`route_next` returns `"research"` or `"orchestrator"`) and the mapping converts it to a node name.
- **compile()** validates the graph and returns a runnable (`app`) with `invoke` and `stream`.

3.4 Fan-out with `Send` (L308–324)

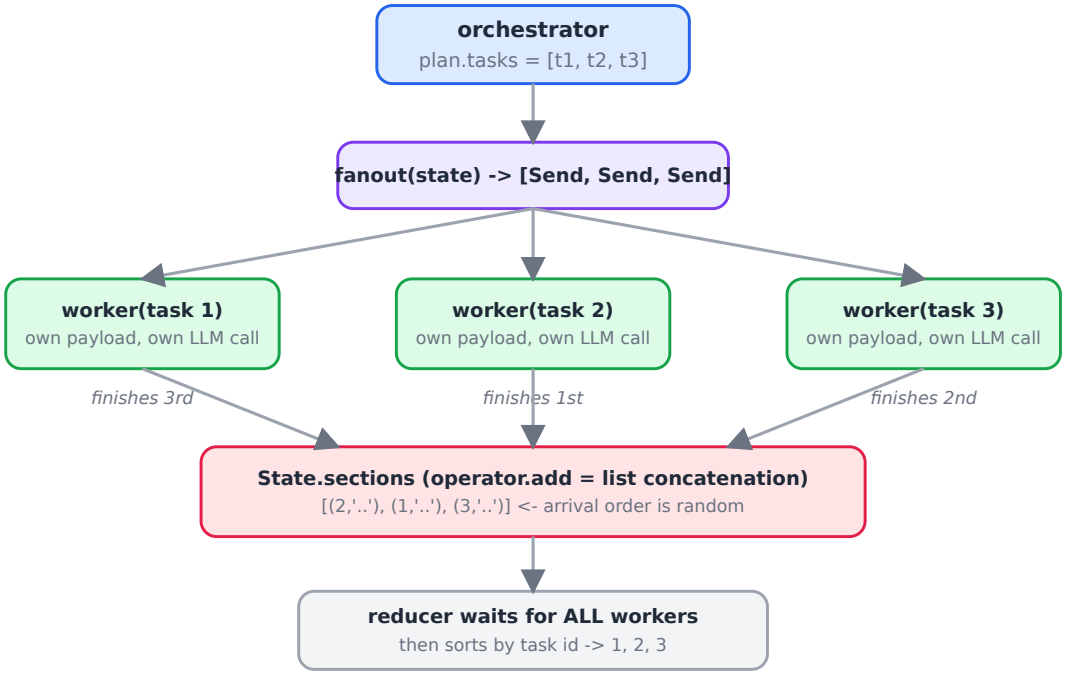


Figure 2. Map-reduce: one Send per task, parallel workers, state reducer collects results, reducer node sorts them.

- Static edges cannot express “run the worker N times” when N comes from the plan at run time. A conditional edge function that returns a **list of** `Send(node, payload)` objects can.
- The payload is that worker's **private input**, not the global state. That is why `fanout` copies everything a worker needs (task, plan, topic, mode, dates, evidence) into it, using `model_dump()` to turn Pydantic objects into plain dicts.
- All workers launched in one step run concurrently. The edge `worker -> reducer` means the reducer starts in the next step, after every worker has finished. This acts as a **barrier**.
- Workers finish in random order. Each section carries its task id, and `merge_content` sorts by id (L399). “Parallel but deterministic output order” is a good one-liner.

3.5 Subgraph (L543–552)

The reducer is its own `StateGraph(State)` with three nodes, compiled and added to the main graph as a single node. Reasons: encapsulation (the main graph shows one “reducer” box), the three steps are strictly sequential, and it can be tested on its own with a hand-made state. It shares the same `State` schema, so keys such as `sections` and `plan` flow in and `final` flows out. Trade-off: for such a small chain you could also put three nodes directly in the main graph; the subgraph is a readability choice, not a necessity.

3.6 invoke vs stream

- `app.invoke(state)` runs to the end and returns the final state.
- `app.stream(state, stream_mode="updates")` yields `{node_name: partial_update}` after each node. `stream_mode="values"` yields the full state after each step. The Streamlit app uses streaming to show progress.
- No **checkpointer** is configured, so a crash in node 9 loses nodes 1–8. Adding a SQLite or in-memory checkpointer would allow resume and replay (see Q&A).

4. Node-by-node walkthrough

For each node: purpose, key decisions, and what can go wrong. Line numbers refer to `bwa_backend.py`.

4.1 Router (L136–174)

Purpose: decide up front whether the topic needs web research, and produce search queries if so.

- The LLM fills `RouterDecision`: `needs_research`, `mode`, `reason`, `queries`, `max_results_per_query`.
- The prompt defines the modes by example ("weekly", "latest", pricing, policy) and demands *scoped* queries.
- `recency_days` is derived in Python from the mode (L159–164): open_book 7, hybrid 45, else 3650.
- `route_next` is the conditional edge: `needs_research` true goes to `research`, otherwise straight to `orchestrator`.

Why it exists: cost and quality. Searching for "Self Attention" wastes API calls; writing "this week's AI news" without searching produces hallucinated news.

Risks: misclassification (a free model may label a news topic closed_book). `max_results_per_query` is returned but never used by `research_node` (it hard-codes 6). Field order note: `reason` comes *after* `mode` in the schema, so it explains the decision rather than helping make it; putting it first can improve classification slightly.

4.2 Research (L176–255)

Purpose: turn queries into a clean list of `EvidenceItem` (title, url, published date, snippet).

1. `_tavily_search` (L179–199) calls Tavily, normalises fields, and returns `[]` if the key is missing or anything throws.
2. Up to 10 queries × 6 results are collected (L222–225).
3. An LLM "research synthesizer" (L230–242) deduplicates, keeps authoritative sources and normalises dates to ISO.
4. Python dedups by URL (L244–248).
5. For **open_book only**, a hard filter keeps items with a parseable date within the window (L250–253).

Design reasoning: the filter is code, not a prompt, because "only last 7 days" must be reliable.
Weaknesses: the LLM step sends all raw results (this was 23,209 prompt tokens in a real run), burns output tokens, and can hallucinate dates; undated items are discarded in open_book; errors are swallowed silently. Sections 10 and 11 cover the fix.

4.3 Orchestrator / planner (L260–302)

Purpose: produce the `Plan`: title, audience, tone, blog kind and 5–9 tasks, each with goal, 3–6 bullets and a word target.

- The prompt acts as a spec: hard requirements, a quality bar (must include e.g. a code sketch, failure modes, cost considerations) and grounding rules per mode.
- Only the first 16 evidence items are shown (L294) to bound prompt size.
- **Belt and braces:** the prompt says "open_book means news_roundup" *and* code forces it afterwards (L299–300). Never trust the model alone for a rule you can enforce.

Why plan first? (1) Writers can run in parallel because the structure is fixed. (2) Non-overlapping bullets reduce repetition. (3) You can inspect or edit the plan before spending tokens on writing. (4) Easier to evaluate: check number of sections, word totals, flags. **Trade-off:** workers do not see each other's text, so transitions can be uneven.

Known crash: if the structured call returns `None`, line 300 raises `AttributeError` (section 11, story 2).

4.4 Fanout (L308–324)

Described in 3.4. Design point: it also asserts `plan is not None`, and passes **all** evidence to **every** worker, which is simple but sends the same URLs N times (cheap per worker, since only title, URL and date are

rendered).

4.5 Worker (L329–389)

Purpose: write one Markdown section. This is the node that runs N times.

- Rebuilds typed objects from the payload (`Task(**payload["task"])`) which re-validates the data.
- The system prompt contains: coverage rule (all bullets, in order), length (±15%), format (starts with `## Title`), a **scope guard** (news roundups must not drift into tutorials about scraping or RSS), a **grounding policy**, and a code rule.
- Evidence is rendered as plain lines `title | url | date` (L357–360) and labelled "ONLY cite these URLs", a **URL whitelist**.
- The honest-failure phrase "Not found in provided sources." gives the model a safe exit instead of inventing facts.
- Returns `{"sections": [(task.id, section_md)]}`, a single-item list that `operator.add` appends.

Risks: word count is a request, not enforced; links depend on model obedience; `.content.strip()` assumes a string (L387).

4.6 Reducer subgraph (L395–541)

- **merge_content**: pure Python. Sort by task id, join with blank lines, prepend `# {blog_title}`. No LLM, no failure modes.
- **decide_images**: LLM reads the merged blog and returns `GlobalImagePlan`. Section 6.
- **generate_and_place_images**: Python loop that generates, saves and substitutes, then writes `generated_markdowns/<slug>.md`.

5. Schemas and structured output

5.1 Why Pydantic (L35–90)

- **Contract:** the schema is simultaneously documentation, validation and the JSON schema sent to the model.
- **Constraints in the type:** `bullets: List[str] = Field(min_length=3, max_length=6)` rejects sloppy plans; `Literal[...]` restricts `mode`, `blog_kind` and image `size` to allowed values; defaults (`default_factory=list`) make optional fields safe.
- **Serialisation:** `model_dump()` converts objects to dicts so they can travel through `Send` payloads, and `Task(**dict)` rebuilds them.

Schema	Used by	Purpose and notable fields
`RouterDecision`	router	needs_research, mode (Literal), reason, queries, max_results_per_query
`EvidenceItem` / `EvidencePack`	research	title, url, published_at (ISO or null), snippet, source; Pack is the list wrapper needed because a model returns one object
`Task`	planner, worker	id, title, goal, bullets (3–6), target_words, flags requires_research / citations / code
`Plan`	planner, worker, reducer	blog_title, audience, tone, blog_kind, constraints, tasks
`ImageSpec`	image planner	placeholder, filename, alt, caption, prompt, size, quality
`GlobalImagePlan`	decide_images	md_with_placeholders + list of ImageSpec

5.2 How `with_structured_output` works

1. LangChain converts the Pydantic class to a JSON schema.
2. It sends that schema to the provider as a **tool definition** (function calling) or as a **response_format** (JSON-schema mode), depending on method and provider.
3. The model returns arguments that match the schema.
4. LangChain parses and validates them into a Pydantic object.

5.3 Failure modes (know all four)

Failure	Symptom	Mitigation
Model skips the tool call	Function-calling mode can return `None` silently	Retry wrapper; pin a model that supports tools
Output truncated	`finish_reason="length"`, `LengthFinishReasonError`	Smaller prompt, bigger `max_tokens`, limit reasoning, simplify schema
Validation error	e.g. 2 bullets when 3 are required	Retry with the error fed back to the model
Provider lacks the feature	400 error or ignored schema	Choose method per provider; test with a tiny call first

6. The image pipeline

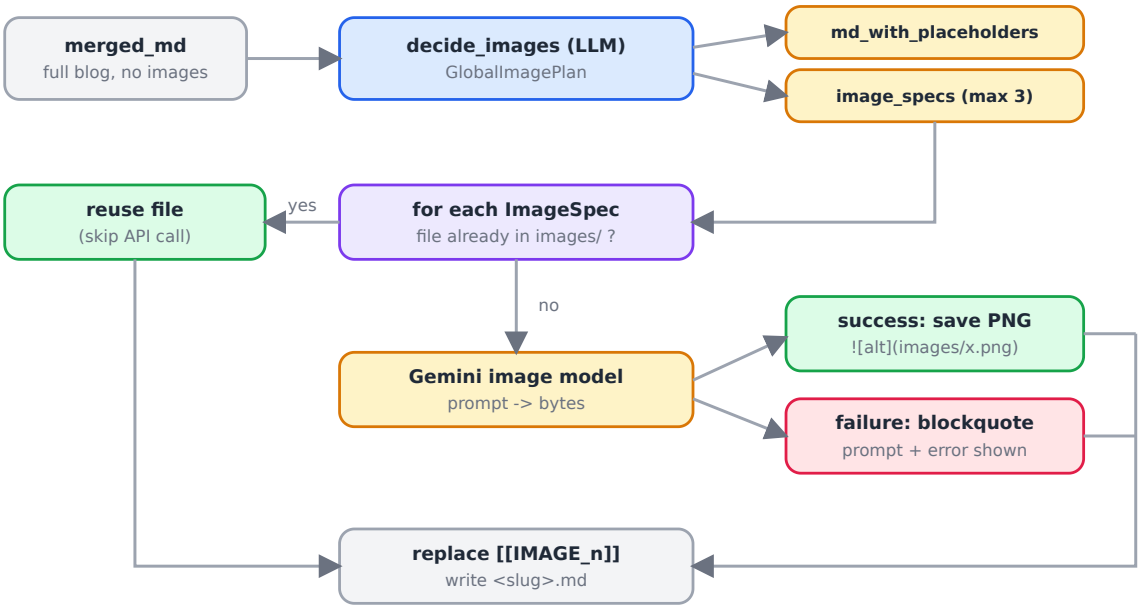


Figure 3. Image flow inside the reducer subgraph.

6.1 Step by step

1. **Decide (L405–440).** The LLM sees the full merged blog and returns `GlobalImagePlan`: the Markdown with `[[IMAGE_1]]` style placeholders inserted, plus up to 3 `ImageSpec`s. Rules in the prompt: only images that materially help understanding, prefer technical diagrams with short labels, and if none are needed return the input unchanged with an empty list.
2. **Generate (L443–488).** `_gemini_generate_image_bytes` calls `gemini-2.5-flash-image` with `response_modalities=["IMAGE"]` and a relaxed safety threshold for dangerous-content false positives. It handles SDK differences (`resp.parts` vs `resp.candidates[0].content.parts`) and extracts `inline_data.data` bytes.
3. **Place (L498–541).** For each spec: if `images/<filename>` exists, reuse it; otherwise generate and save. On success replace the placeholder with `![alt](images/file)` followed by an italic caption line. On failure replace it with a blockquote containing the caption, alt text, prompt and error. Finally write the `.md`.

6.2 Design decisions to defend

- **Placeholders as anchors:** separates *deciding* (LLM, creative) from *doing* (Python, deterministic). A failed API call can only affect one placeholder, never the whole document.
- **Image step after merge, not inside workers:** the planner sees the whole article, so it can pick the 3 most valuable places and avoid duplicates.
- **Hard cap of 3:** bounds cost and latency.
- **Graceful fallback:** the document stays usable; the blockquote prompt lets a human generate the image elsewhere.
- **Relative paths in Markdown** (`images/x.png`) keep the file portable when zipped with the folder.

> **BE READY TO DISCUSS**
> The decide step asks the LLM to *return the whole blog again* with placeholders added. That costs as many output tokens as the article, risks truncation, and can silently alter text or citation links. A better design is to return only insertion anchors (e.g. “after heading X”) and insert in Python. See weak point 6.

7. Grounding, prompts and safety

7.1 Layered defence against hallucination

No single technique is reliable, so the project stacks several. Notice that the strongest layers are in **code**.

#	Layer	How it works in the project
1	Router decides if facts can go stale	Fresh topics get research; evergreen ones do not need it
2	Evidence whitelist	Workers may cite only URLs listed in the prompt (L383)
3	Hard recency filter (code)	open_book drops items older than 7 days or undated (L250–253)
4	Honest failure phrase	“Not found in provided sources.” replaces invented claims
5	Scope guard	News roundups must not drift into tutorials
6	Forced `blog_kind` (code)	Planner output corrected after the fact (L299–300)
7	Insufficient-evidence plan	Planner is told to say so transparently when sources are thin

> **PRINCIPLE TO QUOTE**
> “Prompts ask, code enforces.” Anything that must always be true (date windows, section order, file paths, blog kind) should be implemented in Python.

7.2 Prompt design patterns used

- **Role + task:** “You are a senior technical writer...” sets register and vocabulary.
- **Hard constraints in a list:** numbered, testable requirements (5–9 sections, 3–6 bullets, 120–550 words).
- **Mode-conditional rules:** the same prompt behaves differently for closed_book / hybrid / open_book.
- **Explicit output format:** “Output ONLY the section Markdown starting with `## Title`” makes merging trivial.
- **Context packaging:** the human message carries the plan metadata (title, audience, tone, constraints) so every parallel worker writes in a consistent voice.
- **Low temperature (0.3):** favours reliable structure while keeping prose natural.

7.3 Security and privacy

- **Indirect prompt injection:** web snippets are pasted into prompts. A malicious page could contain “ignore previous instructions”. Mitigations: fence untrusted text, tell the model it is data, strip instructions, and keep the model's power low (it can only write text, not run tools).
- **Secrets:** API keys come from environment variables (`.env`), never hard-coded. Make sure `.env` is in `.gitignore`.
- **Rendering:** Streamlit calls use `unsafe_allow_html=False` so generated Markdown cannot inject HTML.
- **Content safety for images:** a safety setting is passed to Gemini; failures are caught and reported rather than crashing.

8. Providers, cost and reliability

8.1 Provider history and what it teaches

The project has used OpenAI-style models, an OpenRouter free router (`ChatOpenAI` with a custom `base_url`), and now Groq (`ChatGroq`, `gpt-oss-20b`). Lessons you can state:

- **Free tiers are constrained:** low requests-per-minute and tokens-per-minute (TPM). A parallel fan-out is exactly the pattern that triggers 429 errors.
- **Models differ in structured-output support** and some are **reasoning models** whose hidden thinking consumes the output budget.
- **Keep LLM construction in one place** (L121–132). Swapping providers is then a 5-line change, which is why the old config is left commented out.

8.2 What one run costs

Resource	Per run	Notes
LLM calls	about 9–13	router 1 + research 1 + planner 1 + N workers (5–9) + image planner 1; research is skipped in closed_book
Tavily searches	up to 10	one per query, 6 results each
Image generations	up to 3	skipped if file exists, or if the planner returns none
Largest prompts	research extractor, image planner	raw search results; the full blog
Largest outputs	image planner, workers	the planner re-emits the whole article

> **IMPORTANT**
> The Streamlit app currently runs the graph twice per click (section 9.4), so every number above is doubled in practice.

8.3 Reliability techniques (current and recommended)

Technique	Status
Fallback instead of crash for images	Implemented (try/except per image)
Default values for optional schema fields	Implemented
Deterministic ordering of parallel output	Implemented
Retry with backoff on 429 / transient errors	Recommended: `max_retries` on the client
Cap concurrency of parallel workers	Recommended: `config={"max_concurrency": 3}`
Retry on invalid structured output, then raise a clear error	Recommended (section 11)
Checkpointing to resume a failed run	Recommended: LangGraph checkpointer
Tracing and evaluation	Recommended: LangSmith or logging of prompts, token usage and finish reasons

9. Streamlit frontend

Short version for interviews: the frontend is a thin layer that collects a topic, runs the compiled graph, streams node-by-node progress, and displays the plan, evidence, Markdown preview and images. All intelligence is in `bwa_backend.py`, which the UI imports as `from bwa_backend import app` (L18).

9.1 The Streamlit execution model

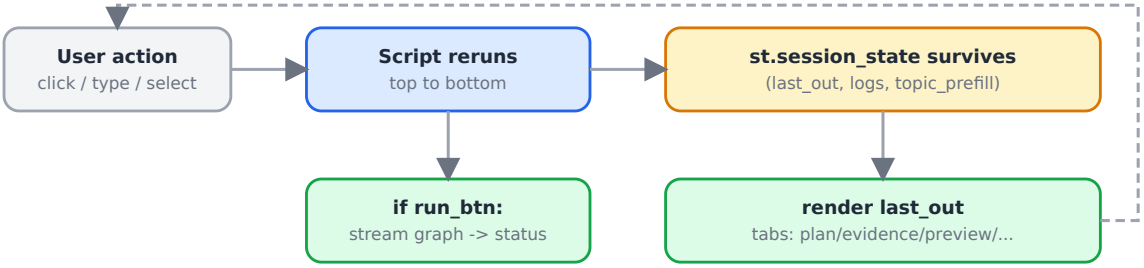


Figure 4. Streamlit reruns the whole script on every interaction.

- Streamlit executes the script **top to bottom on every interaction** (button click, text change, radio change). Ordinary variables reset on each rerun.
- `st.session_state` is the dictionary that survives reruns. The app stores the latest graph output in `st.session_state["last_out"]` (L256–257, L324), so the preview stays visible after other clicks.
- That is why the structure is: sidebar inputs, then `if run_btn:` (run the graph), then `out = st.session_state.get("last_out")` and render.

9.2 Layout and features

Feature	Implementation
Inputs (sidebar)	`st.text_area` topic, `st.date_input` as-of date (default today), primary button (L196–203)
Initial graph state	A dict with every State key initialised (L276–290), required so the graph starts with consistent types
Live progress	`st.status` container, writes the node name when it changes; `st.empty().json(summary)` shows mode, queries, evidence count, tasks, images, sections done (L292–320)
Five tabs	Plan (table of tasks via pandas), Evidence (dataframe), Markdown Preview, Images, Logs (L260–262)
Custom Markdown renderer	`render_markdown_with_local_images` (L105–158): `st.markdown` cannot show local relative images, so a regex splits the text into markdown and image chunks, shows images with `st.image`, and promotes the italic line after an image to its caption
Downloads	`st.download_button` for the `.md`, and a ZIP bundle (markdown + images) built in memory with `zipfile` and `BytesIO` (L33–53, L411–425)
Past blogs	Sidebar radio of the newest 50 `.md` files in `generated_markdowns/`, with titles read from the first `#` heading; “Load” puts the text in `last_out` (L164–186, L205–246)
Robust data handling	Plan and evidence may be Pydantic objects (from a live run) or dicts/empty (loaded blog), so the code checks `hasattr(obj, "model_dump")` (L338, L381)

9.3 Streaming design (L56–90)

`try_stream` is a generator that yields `("updates", step)` items as the graph runs, then a `("final", state)` item. `extract_latest_state` merges each update into a running dictionary used for the progress summary. A generator keeps the UI loop simple: `for kind, payload in try_stream(...)`.

9.4 Frontend weak points

Issue	Detail and fix
Graph runs twice (L62–65)	After `stream()` finishes, the code calls `invoke()` again to get the final state, re-executing every LLM, search and image call. Fix: stream once with `stream_mode="values"` and use the last chunk as the final state.
Errors swallowed and retried (L67–68, L76–77)	`except Exception: pass` hides the real error and starts another full run; the user may wait through up to three executions before an error surfaces. Fix: catch once, show `st.error` with the message.
Image path resolution (L100–102)	Markdown contains `images/x.png`, resolved relative to the *current working directory*, but images are saved under `generated_markdowns/images`. The preview can show “Image not found” unless the app is launched from that folder. Fix: resolve against `OUTPUT_DIR`. (Check this on your machine.)
ZIP paths (L41, L52)	`arcname=str(p)` stores the full absolute path inside the zip. Fix: use `p.relative_to(OUTPUT_DIR)`.
Blocking UI	A run takes minutes and blocks the script. Acceptable for a prototype; production would use a background job and polling.
Progress counter	In updates mode each parallel worker's update overwrites `sections`, so “sections_done” shows 1, not N. Fix: count worker updates.

10. Weak points and honest limitations

Interviewers often ask “what would you improve?”. Use this pattern: **name the issue, state the impact, give the fix, and say how you would verify it.** Showing that you found these yourself is a strength.

#	Where	Issue and impact	Fix
1	Frontend `try_stream` L56–80	Whole graph executes twice per click; errors hidden. Doubles cost and rate-limit pressure.	One `stream(stream_mode="values")`; surface errors.
2	`_tavily_search` L179–199	Missing key or any exception returns `[]` silently; the blog is generated without evidence and no one is told.	Log or raise; show a UI warning.
3	Research LLM step L230–242	Tens of thousands of input tokens and thousands of output tokens just to copy fields Tavily already returns; can truncate (`LengthFinishReasonError`) or hallucinate dates.	Normalise, dedup, truncate snippets and filter in Python. No LLM call.
4	Recency filter L250–253	Drops undated items, so open_book can end with zero evidence.	Use Tavily news topic and time range (returns dates); show a warning when thin.
5	Structured calls L151, L279, L299	Function-calling returns `None` when the model skips the tool; `plan.blog_kind` then crashes.	Retry wrapper with `include_raw=True` and clear error.
6	`decide_images` L417–440	LLM rewrites the entire article: high output cost, truncation risk, silent edits, can break links.	Return only anchors and ImageSpecs; insert in Python; verify placeholders exist.
7	Image cache L519–521	Skip-if-exists is keyed by LLM-chosen filename, not by content; two blogs both using `attention_diagram.png` would reuse the wrong image.	Include a hash of the prompt (or blog slug) in the filename.
8	ImageSpec L83–84 vs L458	`size` and `quality` are collected but never passed to Gemini.	Use them or remove them.
9	Citations in worker L342–345	Links depend on model obedience; no guaranteed Sources section.	Reducer appends a deterministic Sources list from evidence.
10	Workers are isolated	Repetition and uneven transitions between sections; all evidence sent to all workers.	Per-section evidence; optional final editor pass (costs one more call).
11	Word counts	LLMs count poorly; target is a request only.	Measure and accept a range, or regenerate outliers.
12	Concurrency	N parallel workers can trip provider TPM limits.	`max_concurrency`, retries with backoff.
13	No persistence, tests or evaluation	A failure at the last node loses all work; quality is judged by eye.	Checkpointer, LangSmith traces, a small automated checklist (section count, citations present, no empty sections).
14	Prompt injection	Search snippets are untrusted text inside prompts.	Delimit and label as data; limit model capabilities; filter obvious instructions.
15	Filename collisions L508	Same title overwrites the previous `.md`.	Append a timestamp or short hash.

11. Improvements and debugging stories

HONEST NOTE
The changes below were made and tested in the **notebook version** (`4_bwa_research_fine_tuned_v2.ipynb`) during development. They are **not yet in** `bwa_backend.py`. Before the interview, port them if you can. If you cannot, present them accurately as "improvements I prototyped and plan to merge".

11.1 What changed in version 2

Change	Reason
Research without an LLM call	Tavily already returns title, url, content and (for news) a date. Dedup, 400-character snippets, date parsing and the recency filter are done in Python; evidence is capped at 12. Saves roughly 23k input and up to 8k output tokens per run and removes a failure point.
`langchain-tavily` with `topic="news"`	Provides publication dates so the open_book filter works; the old community tool is deprecated.
`structured_invoke()` helper	Uses `method="function_calling"`, `include_raw=True`, retries up to 3 times with the validation error fed back, and raises a clear error that includes `finish_reason`.
Fewer queries and results	At most 5 queries and the router's `max_results_per_query` clamped to 3–5, so the unused field is finally used.
Evidence to all workers + Sources section	Guarantees links in the output even if the model forgets inline citations.
Client settings	`max_tokens` cap, `max_retries=3`, `max_concurrency=3`, reasoning effort set to low, `_text()` helper for list-type message content.

11.2 Three debugging stories (STAR format)

STORY 1: THE RESEARCH NODE FAILED WITH LENGTHFINISHREASONERROR
Situation: the graph crashed in `research` when parsing the structured response.
Task: find whether the schema, the prompt or the model was at fault.
Action: read the usage metadata in the traceback: 23,209 prompt tokens and 8,192 completion tokens, all of them *reasoning* tokens, so the model never wrote any JSON. Suggested fixes were to raise `max_tokens` and drop fields; I rejected dropping `published_at` because the recency filter depends on it, and rejected the token increase because a reasoning model can consume any budget.
Result: removed the LLM step entirely and did the work in Python.
Lesson: read the evidence (token counts), and do not use an LLM for a transformation that plain code can do.

STORY 2: THE PLANNER RETURNED NONE
Situation: `AttributeError: 'NoneType' object has no attribute 'blog_kind'` at the line that forces the blog kind.
Action: traced it back: function-calling structured output yields `None` instead of raising when the model does not produce a valid tool call (skipped call, truncation, free models that ignore tools). Wrote a wrapper using `include_raw=True` that retries, feeds back the parsing error, and raises an informative error.
Lesson: never assume an LLM call returned what you asked for; validate at every boundary.

STORY 3: THE FINAL MARKDOWN HAD NO LINKS
Action: checked each stage in order: did research return evidence (search key, date filter dropping undated items)? Did workers receive the evidence (an optimisation had hidden it from most sections)? Did the model choose to cite? The reducer had no safeguard.
Result: evidence goes to every worker and the reducer appends a deterministic Sources section.
Lesson: prompts ask, code enforces.

12. Interview Q&A bank

Answers are written so you can say them in 20–40 seconds. Put them in your own words; do not memorise them. Questions marked *follow-up* are what a good interviewer asks next.

A. Project and motivation

Q1. Walk me through your project in two minutes.

Use the 2-minute version from section 0: topic in; router decides research mode; Tavily evidence if needed; planner produces a structured Plan; workers write sections in parallel via LangGraph `Send`; reducer merges in task order; an image planner inserts placeholders; Gemini generates images; Python writes the Markdown. Streamlit shows progress and results.

Q2. Is this an agent?

It is an agentic workflow. I fixed the steps and used LLMs for judgement inside them (routing, planning, writing, choosing where images help). A free-roaming tool-calling agent would be more flexible but harder to bound in cost and behaviour, and blog writing has known steps.

Follow-up: when would you use a real agent? When the steps cannot be known ahead of time, e.g. open-ended research or debugging.

Q3. Why three modes (closed_book, hybrid, open_book)?

Freshness is the main risk for factual accuracy and search costs money. Evergreen topics skip research; hybrid mixes concept explanation with current examples; open_book is news-style, where every claim must be backed by a source from the last 7 days.

Q4. What is decided by the LLM and what by code?

LLM: mode and queries, outline, section text, image placement and prompts. Code: recency windows, the date filter, forced `blog_kind`, URL dedup, section ordering, file names, image generation, fallbacks and file writing. Rule: if something must always hold, enforce it in code.

Q5. Why generate a plan before writing?

The plan is a contract that makes parallel writing possible, reduces overlap between sections, can be inspected before spending tokens, and is easy to evaluate. The cost is that workers cannot see each other's text.

B. LangGraph

Q6. What is the State and how is it updated?

A `TypedDict` shared by all nodes. A node returns a partial dict and LangGraph merges it. By default a key is overwritten; a key annotated with a reducer function (like `operator.add`) is combined instead.

Q7. Why `Annotated[List[...], operator.add]` on `sections`?

Many workers write to `sections` in the same step. Without a reducer LangGraph cannot choose between concurrent writes and raises an error. `operator.add` concatenates lists, so every worker's single-item list is appended.

Q8. Explain `Send`. Why not ordinary edges?

The number of workers equals the number of plan tasks, known only at run time. A conditional edge function returns a list of `Send('worker', payload)`, creating one parallel task per item, each with its own payload. Static edges cannot express a variable fan-out.

Q9. How does the reducer know that all workers are done?

`add_edge('worker', 'reducer')`: nodes launched in the same step finish before the next step begins, so the reducer runs once after all workers complete.

Q10. Workers finish in random order. How do you keep the blog in order?

Each worker returns `(task.id, markdown)` and `merge_content` sorts by id. Parallel execution, deterministic output.

Q11. Why is the reducer a subgraph?

Encapsulation and testability: three sequential steps appear as one node, and the subgraph can be run alone with a hand-made state. It shares the `State` schema so keys flow through. It is a readability choice rather than a technical necessity.

Q12. What is the difference between `invoke` and `stream`?

`invoke` returns only the final state. `stream` yields updates per node (`updates`) or the full state each step (`values`), which is what powers the live progress panel.

Q13. What would a checkpointer add?

State is saved after each step, so a failed run can resume from the last successful node instead of repeating paid LLM calls; it also enables human-in-the-loop (pause to approve the plan) and replay for debugging.

C. LLM engineering

Q14. How does structured output work?

The Pydantic model becomes a JSON schema. It is sent as a tool definition (function calling) or response format. The model replies with matching arguments, and LangChain validates them into an object.

Q15. What can go wrong with structured output?

The model may skip the tool call (you get `None`), the response may be truncated (`finish_reason='length'`), validation may fail (too few bullets), or the provider may not support the method. I handle it with retries, a larger budget, simpler schemas and models that support tools.

Follow-up: how would you test it? Call each structured step 20 times and measure the failure rate.

Q16. Why Pydantic instead of asking for JSON in the prompt?

Schema enforcement and validation are automatic, constraints (`min_length`, `Literal`) are declarative, the same class documents the contract, and `model_dump()` gives serialisable data for `Send`.

Q17. How do you reduce hallucinations?

Seven layers (section 7.1): router, evidence whitelist, hard recency filter, honest failure phrase, scope guard, forced blog kind, and a transparent plan when sources are thin. The strongest ones are enforced in code.

Q18. What is prompt injection and does it affect you?

Untrusted text containing instructions. Search snippets are pasted into prompts, so yes. I would delimit them as data, instruct the model to ignore instructions inside them, and keep the model's power low. Here it can only produce text, which limits the damage.

Q19. What are reasoning tokens and how did they affect you?

Hidden thinking tokens some models generate before the answer, billed and counted toward the completion limit. In one failure all 8,192 completion tokens were reasoning, so no JSON was produced. Fix: remove the unnecessary LLM call, limit reasoning effort, or use a non-reasoning model.

Q20. How do you handle rate limits and failures?

Client retries with backoff, a cap on parallel workers, smaller prompts, graceful fallbacks (image failures become a placeholder block) and clear error messages. Next step: checkpointing.

Q21. How would you evaluate the quality of the blogs?

Automatic checks: number of sections equals the plan, no empty sections, citations present and every URL is from the evidence list, word counts in range. Human or LLM-as-judge checks for factual accuracy and coherence on a fixed set of 10–20 topics, compared across versions.

Be honest: I currently judge quality by reading outputs; evaluation is the next improvement.

Q22. Why temperature 0.3?

Structured steps need consistency; prose still benefits from slight variation. For the router I would go lower still (0–0.1).

Q23. How would you reduce cost?

Remove the research LLM call, skip research for evergreen topics, cap queries and results, avoid re-emitting the whole blog in the image step, send only relevant evidence to each worker, cache images and search results, and use a smaller model for routing.

D. Research and images

Q24. Why Tavily and why not a vector database?

The task needs *fresh* web information, not a private document collection. Tavily returns cleaned page text designed for LLMs. A vector database would make sense for a fixed knowledge base.

Q25. How do you ensure recent sources?

In open_book mode a code filter keeps only items with a parseable date inside the 7-day window (and a cutoff computed from `as_of`). The weakness is that undated items are dropped; using Tavily's news topic gives dates.

Q26. Why does the image step use placeholders?

It separates deciding from doing. The LLM chooses where images go and writes prompts; Python generates files and substitutes placeholders. One failed image affects only its placeholder, not the document.

Q27. What happens if image generation fails?

The exception is caught for that image and the placeholder is replaced by a blockquote containing the caption, alt text, prompt and error. The blog is still complete and the user can generate the image elsewhere.

Q28. Is your image caching correct?

Not fully. It skips generation when `images/<filename>` exists, but the filename is chosen by the LLM, so two blogs could collide and reuse the wrong image. Fix: include a hash of the prompt or a blog slug in the filename.

Good example of showing self-awareness.

Q29. What would you change in the image design?

Stop asking the LLM to return the whole blog. Return only anchors (after which heading) plus specs, and insert placeholders in Python. This cuts output tokens, avoids unintended edits and lets me validate anchors.

E. Streamlit

Q30. How does Streamlit run your code?

The script reruns top to bottom on every interaction. Normal variables reset; `st.session_state` persists. That is why the latest result is stored in `session_state['last_out']` and rendered after the run block.

Q31. How does the progress display work?

The graph is streamed. Each update is merged into a running dictionary, the node name is written to an `st.status` box, and a summary (mode, evidence count, sections done) is shown with `st.json` in an `st.empty()` placeholder.

Q32. Why a custom Markdown renderer?

`st.markdown` does not display local relative image paths. The renderer splits the Markdown with a regex into text and image chunks, shows images with `st.image` and promotes the italic line after an image to a caption.

Q33. What is wrong with `try_stream`?

It streams the graph and then calls `invoke()`, so everything executes twice, and it swallows exceptions then retries, which can run the graph up to three times before the real error appears. Fix: stream once with `stream_mode='values'`, take the last chunk as the result, and show the exception.

This is a great answer: it shows you can find cost bugs.

F. Reflection and behavioural

Q34. What was the hardest bug and how did you debug it?

Choose story 1 or 2 from section 11 and tell it in STAR format. Emphasise reading the evidence (token usage, `finish_reason`) before changing code.

Q35. You used AI tools to write code. What did you decide yourself?

Be upfront. Say AI helped with syntax, while you decided the architecture: workflow vs agent, the three modes, plan-then-parallel-write, which steps are LLM and which are code, the grounding policy, the image placeholder design, the schemas, and how failures are handled. Then prove it by explaining any function on request.

Never claim you typed every line; interviewers value honesty plus understanding.

Q36. What would you do with one more week?

Port version 2 into the backend, add a checkpointer and tracing, build a 10-topic evaluation set, fix the image-step design and the double-run in the UI, and add an optional human approval of the plan.

Q37. How would you scale this to many users?

Run the graph in a background worker queue instead of inside the web request, store results and images in object storage, add per-user rate limiting, cache search and image results, use a provider with higher limits, and persist runs with a checkpointer.

Q38. What are the privacy or security risks?

Prompts and topics go to third-party APIs (Groq, Tavily, Google), so sensitive text should not be submitted; API keys must stay in environment variables; web content is untrusted (prompt injection); generated Markdown is rendered without raw HTML.

Q39. What did you learn?

LLM output must be validated like any untrusted input; deterministic code beats prompts for hard rules; read token usage and finish reasons before blaming the prompt; and cost shows up in architecture (extra LLM calls, duplicated runs), not only in model choice.

13. Cheat sheet

One-line description of every function

Function (lines)	One-liner
`router_node` (L150–171)	LLM classifies topic into a mode and proposes queries; Python sets the recency window.
`route_next` (L173)	Conditional edge: research or straight to planning.
`_tavily_search` (L179)	Calls Tavily and normalises fields; returns an empty list on any failure.
`research_node` (L221)	Collect results, LLM-normalise into EvidenceItems, dedup, and hard-filter by date in open_book.
`orchestrator_node` (L278)	LLM produces the Plan; code forces `news_roundup` for open_book.
`fanout` (L308)	Returns one `Send` per task, each with a full private payload.
`worker_node` (L351)	Writes one grounded section and returns `(id, markdown)`.
`merge_content` (L395)	Sort sections by id and prepend the title.
`decide_images` (L417)	LLM returns markdown with placeholders and up to 3 image specs.
`_gemini_generate_image_bytes` (L443)	Call Gemini image model and extract the inline bytes, handling SDK shape differences.
`generate_and_place_images` (L498)	Generate or reuse files, substitute placeholders, write the `.md`; failures become blockquotes.
`try_stream` (frontend L56)	Generator that streams graph updates then yields the final state (runs the graph twice; see weak points).
`render_markdown_with_local_images` (frontend L105)	Split Markdown and show local images with captions.
`list_past_blogs` (frontend L164)	Newest `.md` files in the output folder for the sidebar.

Ten sentences to land in the interview

1. It is an agentic workflow: fixed graph, LLM judgement inside nodes.
2. The router saves cost by researching only when freshness matters.
3. A Pydantic Plan is the contract between planner and writers.
4. LangGraph `Send` gives dynamic parallel fan-out; `operator.add` merges concurrent writes.
5. Workers finish in random order, so sections carry ids and the reducer sorts.
6. Prompts ask, code enforces: date filter, blog kind, ordering and file I/O are Python.
7. Grounding uses a URL whitelist and an honest failure phrase.
8. Image placeholders separate LLM decisions from deterministic generation, with graceful fallback.
9. Structured output can silently return `None` or truncate, so it needs retries and validation.
10. I know the weak points (double run in the UI, image re-emission, filename-keyed cache) and how to fix them.

Key numbers

3 modes	**7 / 45 / 3650** recency days	**5–9** sections	**3–6** bullets per section
120–550 words per section	**±15%** length tolerance	**≤10** queries × 6 results	**≤3** images per blog
16 evidence items to planner	**20** to each worker	**9–13** LLM calls per run	**0.3** temperature

Before the interview checklist

- Run the app once end to end and keep a sample blog open to show.
- Draw Figure 1 from memory on paper, including state keys.
- Be able to explain `Send`, `operator.add` and the conditional edge without looking.
- Fix or at least reproduce the double-run in `try_stream`; it is a strong talking point.
- Pick two weak points and rehearse the “issue, impact, fix, verify” answer.
- Know your provider, model name and why you switched.